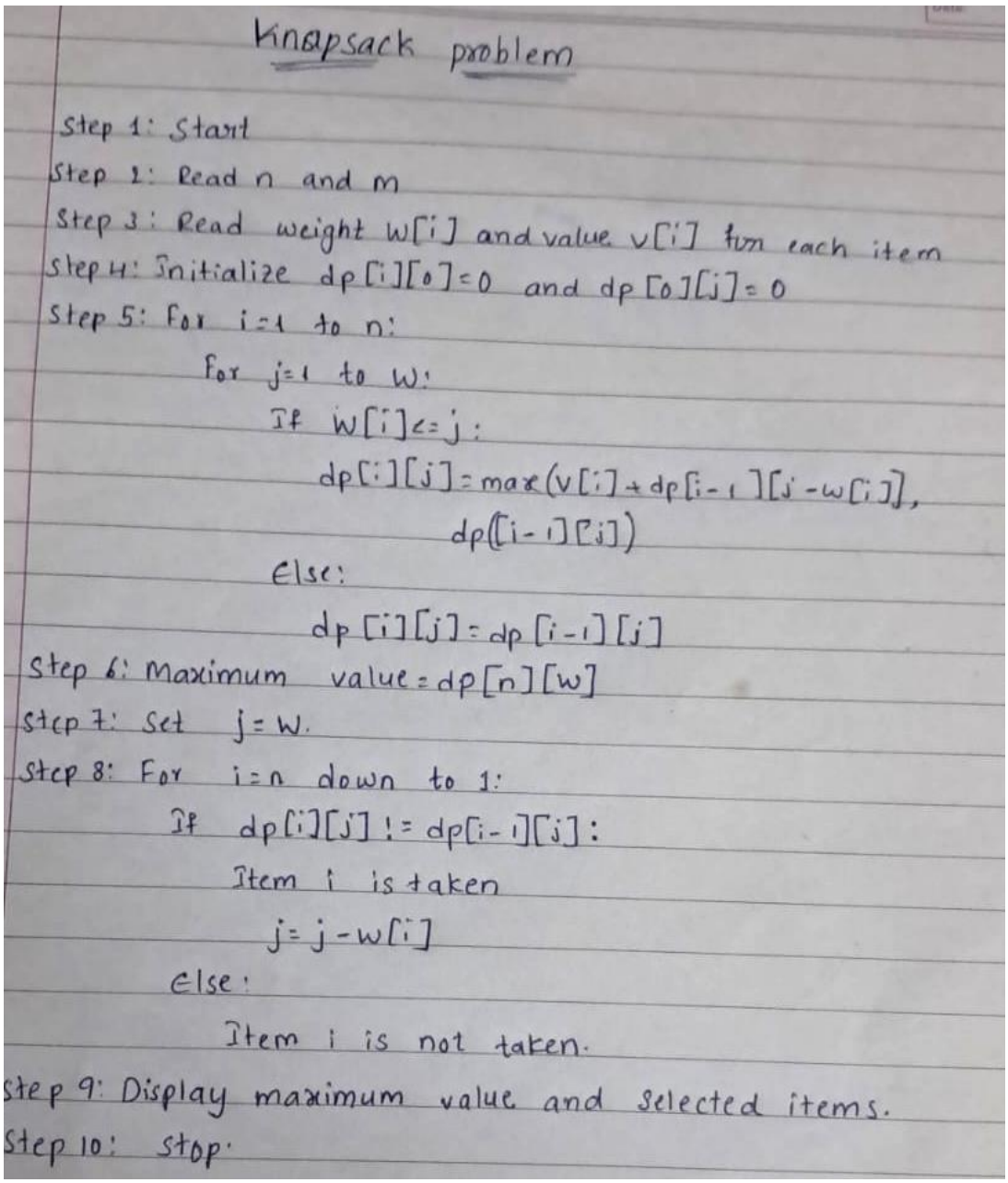
PRACTICAL - 5

AIM :Implementation of Knapsack problem using dynamic programming

ALGORITHM :



Knapsack problem

Step 1: Start
Step 2: Read n and m
Step 3: Read weight w[i] and value v[i] for each item
Step 4: Initialize dp[i][0]=0 and dp[0][j]=0
Step 5: For i=1 to n:
    For j=1 to W:
        If w[i]<=j:
            dp[i][j]=max(v[i]+dp[i-1][j-w[i]],
                dp[i-1][j])
        Else:
            dp[i][j]=dp[i-1][j]
Step 6: Maximum value=dp[n][w]
Step 7: Set j=W.
Step 8: For i=n down to 1:
    If dp[i][j]!=dp[i-1][j]:
        Item i is taken
        j=j-w[i]
    Else:
        Item i is not taken.
Step 9: Display maximum value and selected items.
Step 10: Stop.

PROGRAM :

```
#include<iostream>
using namespace std;

class Knapsack {

private:
    int w[10];       // weights
    int v[10];       // values
    int n;           // number of items
    int W;           // capacity
    int dp[10][100]; // dp table

public:

    // ── Get Input ──
    void getInput() {
        cout << "Enter number of items   : ";
        cin  >> n;
        cout << "Enter knapsack capacity : ";
        cin  >> W;
        for(int i = 1; i <= n; i++) {
```

```
        cout << "Item " << i << " weight and value : ";
        cin  >> w[i] >> v[i];
    }
}

// ── Build dp table ──
void solve() {

    // base case — row 0 and col 0
    for(int i = 0; i <= n; i++) dp[i][0] = 0;
    for(int j = 0; j <= W; j++) dp[0][j] = 0;

    // fill table
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= W; j++) {

            if(w[i] <= j) {
                // take or not take
                dp[i][j] = max(
                    v[i] + dp[i-1][j-w[i]],
                    dp[i-1][j]
                );
```

```
            } else {
                // cannot take item i
                dp[i][j] = dp[i-1][j];
            }
        }
    }
}

// ── Print dp table ──
void printTable() {
    cout << "\nDP Table:" << endl;
    cout << "     ";
    for(int j = 0; j <= W; j++)
        cout << j << "  ";
    cout << endl;

    for(int i = 0; i <= n; i++) {
        cout << "i=" << i << "  ";
        for(int j = 0; j <= W; j++)
            cout << dp[i][j] << "  ";
        cout << endl;
    }
```

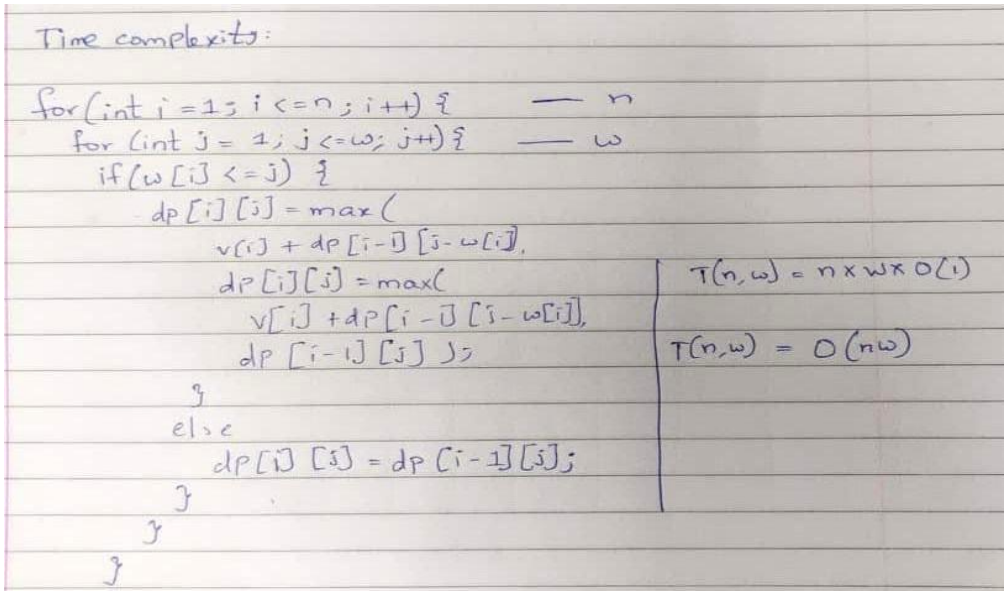
```
}

// ── Find selected items ──
void findItems() {
    cout << "\nSelected Items:" << endl;
    int j = W;
    for(int i = n; i >= 1; i--) {
        if(dp[i][j] != dp[i-1][j]) {
            cout << "Item " << i
                 << " (w=" << w[i]
                 << ", v=" << v[i]
                 << ") TAKEN " << endl;
            j = j - w[i];
        } else {
            cout << "Item " << i
                 << " (w=" << w[i]
                 << ", v=" << v[i]
                 << ") NOT taken " << endl;
        }
    }
}
```

```
    // ── Show result ──
    void showResult() {
        cout << "\nMaximum Value = "
             << dp[n][W] << endl;
    }
};

// ── MAIN ──
int main() {

    Knapsack k;

    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();
     return 0;
}
```

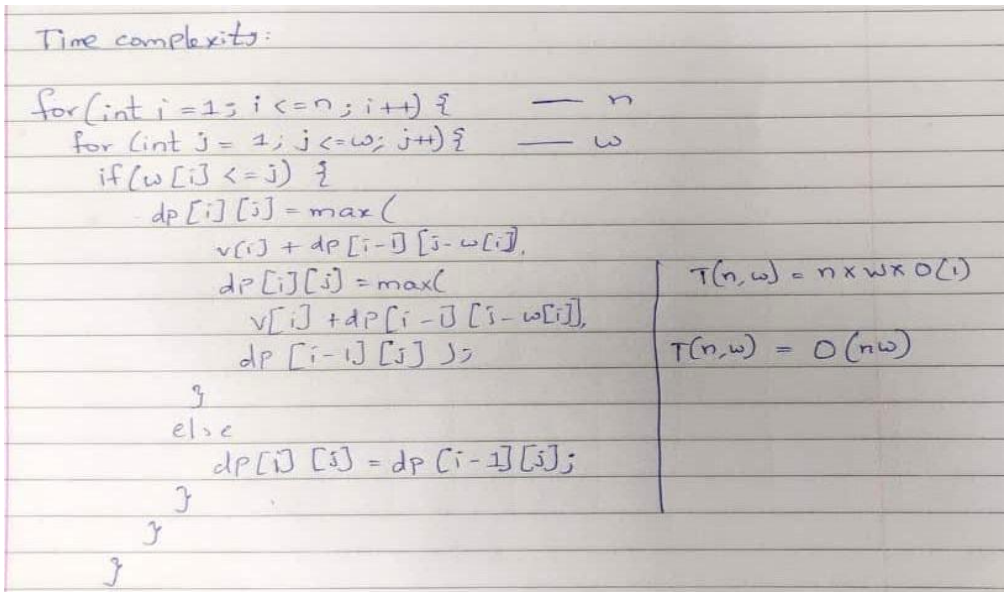
```
Output

Enter number of items   : 3
Enter knapsack capacity : 15
Item 1 weight and value : 3 4
Item 2 weight and value : 2 5
Item 3 weight and value : 7 2

DP Table:
      0  1  2  3  4  5  6  7  8  9  10  11  12  13  14  15
i=0   0  0  0  0  0  0  0  0  0  0  0   0   0   0   0   0
i=1   0  0  0  4  4  4  4  4  4  4  4   4   4   4   4   4
i=2   0  0  5  5  5  9  9  9  9  9  9   9   9   9   9   9
i=3   0  0  5  5  5  9  9  9  9  9  9   9   11  11  11  11

Maximum Value = 11

Selected Items:
Item 3 (w=7, v=2) TAKEN
Item 2 (w=2, v=5) TAKEN
Item 1 (w=3, v=4) TAKEN
```

Time complexity

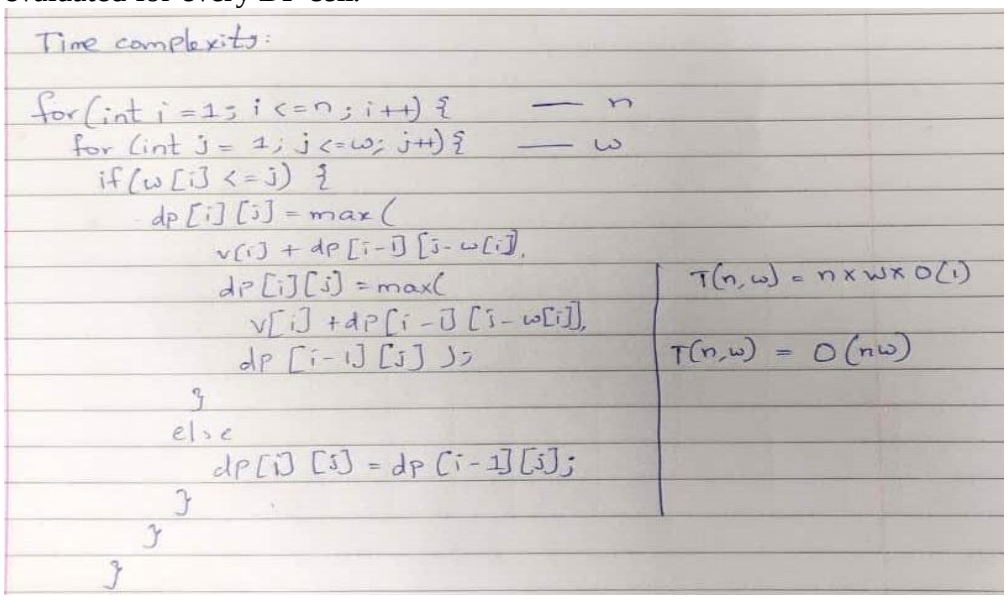
Best case :



Average case : On average, the outer loop executes n times and the inner loop executes W times for each item



Worst case : In the worst case, the same two loops execute completely, and the if condition is evaluated for every DP cell.



Conclusion :

The 0/1 Knapsack problem was successfully implemented using **Dynamic Programming**. The algorithm efficiently finds the maximum possible value within the given capacity, with a time complexity of **O(nW)**.